

C++ Review

Brian

September 27, 2026

Contents

0	Move Semantics and Perfect Forwarding	2
0.1	std::move and std::forward	2
0.2	Universal References vs Rvalue References	3
0.3	std::move on Rvalue Refs, std::forward on Universal Refs	4
0.4	Avoid Overloading on Universal References	6
0.5	Alternatives to Overloading on Universal References	7
0.6	Reference Collapsing	8
0.7	Perfect Forwarding Failure Cases	9
0.8	Moves in Containers	10

0 Move Semantics and Perfect Forwarding

0.1 std::move and std::forward

[EMC++ 23]

- `std::move` is an unconditional cast to an rvalue ref, and `std::forward` only does this cast under specific conditions.

```
1 // std::move
2 template<typename T>
3 typename remove_reference<T>::type&&
4 move(T&& param) {
5     using ReturnT = typename remove_reference<T>::type&&;
6     return static_cast<ReturnT>(param);
7 }
```

- `typename`: explicitly declares that a dependent qualified name is a type.
- `decltype(auto)`: deduce return type and keep refs
- the '&&' in the return type tells us that this returns an rvalue ref.
- We must strip existing references since an rvalue ref to an lvalue ref collapses to an lvalue ref.
- Notice that there is no `remove_cvref`; constness remains.
- Careful!!! this can lead to silent copying.

```
1 struct Widget {
2     Widget() = default;
3     Widget(const Widget&) {std::cout << "copy_ctor\n";}
4     Widget(Widget&&) {std::cout << "move_ctor\n";}
5 }
6 void test () {
7     const Widget w;
8     Widget w2 = std::move(w); // will invoke copy ctor
9 }
```

- A `const Widget&&` cannot bind to `Widget&&`, but can bind to `const Widget&`.
- `std::forward` on the other hand is a conditional cast:

```
1 template <class T>
2 T&& forward(typename std::remove_reference_t<T>& t) noexcept;
```

- Recall that an rvalue ref to an lvalue ref collapses to an lvalue ref; `forward` utilizes this to ensure that function params preserve their original type.
- The act of passing a variable into a function will cause its value category to unconditionally change to an lvalue (since its given a name), even if its type is `T&&`.
- `std::forward` preserves the value category of that object before it was passed; if it was
- concretely, "when `t` is a forwarding reference, this overload forwards the argument to another function with the value category it had when passed to the calling function".
- Forwarding reference: `T&&` is a forwarding reference iff the type `T` must be deduced when `T&&` is called.

0.2 Universal References vs Rvalue References

[EMC++ 24]

- T&& has two meanings: one is an rvalue reference, and the other is either an rvalue or an lvalue (universal reference).

```
1 void f(Widget&& param); // rvalue ref
2 Widget&& var1{Widget()}; // rvalue ref
3 auto&& var2 = var1; // not an rvalue ref (var1 is named, T& &&
  collapses to T&)
4 template<typename T>
5 void f(std::vector<T>&& param); // rvalue ref (only binds to rvalue
  refs)
6 template<typename T>
7 void f(T&& param); // not rvalue ref (universal, can bind to either
  lvalue or rvalue))
```

- Universal references are permitted to bind to rvalue and lvalue references, and any cv qualifiers.
- These are also known as forwarding references as universal references should almost always be passed via `std::forward`.
- If an rvalue is passed into a function via a forwarding reference, it will implicitly be converted into an lvalue as it becomes named within the function body.
- This means that constructing an object here will always invoke the copy constructor; `std::forward<T>` ensures that the original value category that was passed is respected.
- passing an lvalue will remain unchanged, but passing an rvalue will invoke the move constructor.
- Universal references can also be a parameter pack; this is how `emplace_back` is implemented.

```
1 template<class T, class Allocator = allocator<T>>
2 class vector {
3 public:
4     template<class... Args>
5     void emplace_back(Args&&... args); // universal reference; when
      constructing the object in place it will do T(std::forward<Args>(
      args)...)
6 }
```

- Note that since universal references are when any deduction on the type has to be done in the call, `auto&&` is also a universal reference; this can be used in lambda functions to avoid copies.

```
1 auto timeFuncInvocation =
2     [](auto&& func, auto&&... params) { // universal references
3         /* start timer... */
4         std::forward<decltype(func)>(func)(
5             std::forward<decltype(params)>(params)...
6         );
7         /*stop timer and record time*/
8     };
```

- Since we don't have access to T we need this `std::forward<decltype(arg)>` idiom:

- Has essentially the same behaviour as `std::forward<T>`.

consider:

0.3 `std::move` on Rvalue Refs, `std::forward` on Universal Refs

[EMC++ 25]

- `std::move` should only ever be used on rvalue references or if you are certain it is safe to steal resources.
- `std::forward` should be used whenever you have an object that may or may not be moved from.
- I.e. rvalue refs should always be cast to rvalues (with move) when forwarding to other functions; universal references should sometimes be cast to rvalues (with forward) because they're only sometimes rvalues.

```

1 class Widget {
2 public:
3     Widget(Widget&& rhs) // rvalue ref; this can ONLY bind to rvalues
4         : name(std::move(rhs.name)), p(std::move(rhs.p)) {}
5
6     template<typename T>
7     void setName(T&& newName) { //universal reference; can bind to
8         anything
9         name = std::forward<T>(newName); // conditional rvalue cast
10    }
11 private:
12     std::string name;
13     std::shared_ptr<SomeDataStructure> p;
14 };

```

- Technically using `std::forward` on rvalue references will yield the same result as `std::move`:

```

1 void f(std::string&& s) {
2     g(std::move(s)); // okay, as we have rvalue ref
3     g(std::forward<std::string>(s)) // same result, just uglier
4 }

```

- Note that the other way around compiles but may lead to UB

```

1 class Widget {
2 public:
3     template<typename T>
4     void setName(T&& newName) { // universal; can bind to lvalue or
5         rvalue
6         name = std::move(newName); // bad!
7     }
8     /* ... */
9 };
10 Widget w;
11 std::string n = "my_widget";
12 w.setName(n);
13 // using n is UB since it has been moved from

```

- While in the example above overloading setName with two variants (`const string&` and `string&&`) is an option, a function that takes n arguments will require 2^n overloads.
- Functions that take an argument pack would require a seemingly infinite number of overloads; the universal reference ensures that only one implementation is required.

```
1 template<class T, class... Args>
2 shared_ptr<T> make_shared(Args&&... args);
3
4 template<class T, class... Args>
5 unique_ptr<T> make_unique(Args&&... args);
```

- Since using after move is UB, forwarding a universal ref should be the final use of the reference (once its safe to steal its resources).

```
1 template<typename T>
2 void setSignText(T&& text) {
3     sign.setText(text); // use text, but cannot steal yet
4     auto now = std::chrono::system_clock::now();
5     signHistory.add(now, std::forward<T>(text)); // conditionally cast to
6     rvalue
7 }
```

- Returning an object passed as an rvalue or universal reference by value benefits from move or forwarding:

```
1 Matrix operator+(Matrix&& lhs, const Matrix& rhs) {
2     lhs += rhs;
3     return std::move(lhs); // doing return lhs will copy lhs into return
4     value.
5 }
6 // similarly,
7 template<typename T>
8 requires std::convertible_to<T&&, Fraction> // C++20
9 Fraction reduceAndCopy(T&& frac) {
10     frac.reduce();
11     return std::forward<T>(frac);
12 }
```

- However, doing this on local variables returned by value is very bad!

```
1 Widget makeWidget() {
2     Widget w;
3     /* ... */
4     w.doStuff();
5     // return std::move(w); BAD!!
6     return w;
7 }
```

- All modern compilers will perform an optimization called named return value optimization (NRVO) and construct `w` in the place of the return value slot.
- However, one of the requirements is that the returned local object must match exactly the return type in the function signature; since move casts to an rvalue ref, they don't match and NRVO cannot be performed.

0.4 Avoid Overloading on Universal References

[EMC++ 26] Consider:

```
1 std::multiset<std::string> names;
2 void logAndAdd(const std::string& name) {
3     auto now = std::chrono::system_clock::now();
4     log(now, "logAndAdd");
5     names.emplace(name);
6 }
7 /* ... */
8 std::string petName("Alice");
9 logAndAdd(petName); //lvalue string
10 logAndAdd(std::string("Bob")); //rvalue string
11 logAndAdd("AliceBob"); //string literal
```

- This however has inefficiencies when an rvalue is passed, since `name` is an lvalue it gets copied into `names`.
- We can try using a universal reference to fix this:

```
1 template <typename T>
2 void logAndAdd(T&& name) {
3     auto now = std::chrono::system_clock::now();
4     log(now, "logAndAdd");
5     names.emplace(std::forward<T>(name)); // moved for rvalues
6 }
```

- But what happens if we overload `logAndAdd`?

```
1 std::string nameFromIdx(int idx); // return name corresponding to idx
2
3 void logAndAdd(int idx) {
4     auto now = std::chrono::system_clock::now();
5     log(now, "logAndAdd");
6     names.emplace(nameFromIdx(idx));
7 }
8
9 short nameIdx{42};
10 logAndAdd(nameIdx); // ERROR
```

- Since the universal reference binds to everything (and is templated), it takes precedence over the `int` overload.
- We try to `emplace` into a string multiset and thus get an error.
- Universal reference functions are the greediest functions; they will take precedence over any overload that does not exactly match the signature.
- This can lead to some tricky side effects:

```
1 class Person {
2 public:
3     template<typename T>
4     Person(T&& n) : name(std::forward<T>(n)) {}
5
6     Person(const Person& rhs); // copy ctor (compiler generated)
7     Person(Person&& rhs); // move ctor (compiler generated)
8 };
9
```

```

10 Person p("Alice");
11 Person p2(p); //Error!

```

- Even though it looks like we are invoking the copy ctor, the type that p is when we pass it `Person&` matches the universal reference better (no `const`)
- Thus we call the constructor, and attempt to create a string from a `Person` type which errors.

0.5 Alternatives to Overloading on Universal References

[EMC++ 27]

- Pre concepts: use tag dispatch

```

1 template<typename T>
2 void logAndAdd(T&& name) {
3     logAndAddImpl(std::forward<T>(name), std::is_integral<std::
4         remove_reference<T>::type>());
5 }

```

Note we have to add the `std::remove_reference` since if we pass an lvalue, T gets deduced to an lvalue reference and an int reference will return false for `std::integral`.

```

1 template<typename T>
2 void logAndAddImpl(T&&name, std::false_type) { // non-integral
3     /* ... */
4     names.emplace(std::forward<T>(name));
5 }
6
7 void logAndAddImpl(int idx, std::true_type) { logAndAdd(nameFromIdx(idx
8     )); // guaranteed to match correct one}

```

More modern, use `requires` or `if constexpr`

```

1 template<typename T>
2     requires (!std::is_integral_v<std::remove_reference_t<T>>)
3 void logAndAdd(T&& name) { /* ... */}
4
5 // OR
6
7 // match everything to the universal ref
8 template <typename T>
9 void logAndAdd(T&& name) {
10     if constexpr (std::is_integral_v<std::remove_reference_t<T>>) {
11         /* ... */
12     } else {
13         /* ... */
14     }
15 }

```

The trickier case is when there is also inheritance:

```

1 class Person {
2 public:
3     template <typename T>
4     explicit Person(T&& n) : name(std::forward<T>(n)) {}
5     Person(const Person& rhs);

```

```

6     Person(Person&& rhs);
7     ...
8 };
9
10 class SpecialPerson : public Person {
11     SpecialPerson(const SpecialPerson& rhs) : Person(rhs) {}           //
12     // calls Person(T&&), not Person(const Person&)
13     SpecialPerson(SpecialPerson&& rhs) : Person(std::move(rhs)) {} //
14     // same problem
15 };

```

We can require that the forwarded ctor for Person is turned off if we pass a derived type and if it is not constructable from the type passed.

```

1 \begin{cppcode}
2 class Person {
3 public:
4     template <typename T>
5         requires (!std::derived_from<std::decay_t<T>, Person> &&
6                 std::constructible_from<std::string, T&&>)
7         explicit Person(T&& n) : name(std::forward<T>(n)) {}
8     ...
9 };

```

the `std::decay_t<T>` strips any cv type quals and any references.

0.6 Reference Collapsing

[EMC++ 28] consider:

```

1 template<typename T>
2 void func(T&& param); // universal ref
3
4 Widget widgetFactory(); // function returning rvalue
5
6 Widget w; // lvalue
7 func(w); // call func with lvalue, deduced to be Widget&
8 func(widgetFactory()); // call func with rvalue, deduced to be Widget

```

- While references to references are illegal in C++, doing `func(w)` would technically be deduced as `void func(Widget& && param)`
- The references get collapsed
- If either reference is an lvalue ref then it gets collapsed to an lvalue ref; otherwise its an rvalue ref.
- This is how `std::forward` conditionally casts
- Consider:

```

1
2 template<typename T>
3 void f(T&& fParam) {
4     ...
5     someFunc(std::forward<T>(fParam));
6 }
7
8 template<typename T>

```



```

9 T&& forward(typename std::remove_reference_t<T>& param) {
10     return static_cast<T&&>(param);
11 }

```

suppose we pass an lvalue `Widget` into `f` (which deduces to `Widget&`):

```

1 Widget& && forward(typename std::remove_reference_t<Widget&>& param) {
2     return static_cast<Widget& &&>(param);
3 }
4
5 // remove_reference_t gets collapsed (this is there to ensure that T
6 // can never be deduced and must explicitly be written)
7 Widget& && forward(Widget& param) {
8     return static_cast<Widget& &&>(param);
9 }
10
11 // reference collapsing
12 Widget& forward(Widget& param) {
13     return static_cast<Widget&>(param);
14 } // returns lvalue ref

```

For an rvalue, `f` deduces to `Widget`:

```

1 Widget && forward(Widget param) {
2     return static_cast<Widget &>(param);
3 } // returns rvalue ref even though param was an lvalue!

```

0.7 Perfect Forwarding Failure Cases

[EMC++ 30]

- The definition of forwarding is when one function passes its parameters to another function. The second function should receive the exact same objects that the first function received.
- Perfect forwarding extends on this by also ensuring that the value categories and cv qualifiers are also preserved.
- there are cases where this may fail however, specifically when the template arguments for the forwarding function cannot be deduced.

Consider calling a function `f` indirectly by forwarding its args through `fwd`;

```

1 template<typename... Ts>
2 void fwd(Ts&&... params) {
3     f(std::forward<Ts>(params)...);
4 }
5
6 void f(const std::vector<int>& v) {
7     ...
8 }
9
10 fwd(1, 2, 3) // works! rvalues 1, 2, 3 are moved to f and no
11 // temporaries are constructed.
12 f({1, 2, 3}); // converted to std::vector<int>
13 fwd({1, 2, 3}); // error! cannot infer template argument 'T'

```

- When we pass `1, 2, 3` directly to `f`, the compiler compares the args at the call site against the function parameters and sees if there is a trivial conversion.
- When we call `f` through `fwd`, it hides the parameters and thus the compiler cannot figure out what type `1, 2, 3` is.

as well, consider:

```

1 class Widget {
2 public:
3     static const std::size_t MinVals = 28; // MinVal declaration
4     ...
5 };
6
7 // in some other TU
8 template<typename T>
9 void f(std::size_t val) { /*...*/ }
10 f(Widget::MinVals); // works! f(28)
11 fwd(Widget::MinVals); // Linked error (usually)! since Widget::MinVals
    has been folded

```

Forwarding can also fail on overloaded function names:

```

1 void f(int (*pf)(int)); // pf = "processing function"
2 int processVal(int value) { /*...*/ }
3 int processVal(int value, int priority) { /*...*/ }
4
5 f(processVal); // fine; compiler figures out to call the process(int)
    overload
6 fwd(processVal); // Error! ambiguous overload

```

This is exactly the same as the initializer braces situation; calling through `fwd` hides the function signature of `f` and the compiler cannot deduce which overload to call.

Same thing happens with templates:

```

1 template<typename T>
2 T workOnVal(T param) { /*...*/ }
3 fwd(workOnVal); // Error!

```

We can fix this by manually casting to the function we want:

```

1 using ProcessFuncType = int (*)(int);
2
3 ProcessFuncType ProcessValPtr = processVal;
4 fwd(processValPtr);
5 fwd(static_cast<ProcessFuncType>(workonVal));

```

0.8 Moves in Containers

[Josuttis Ch. 4]